

Analisis Penggunaan Asersi dan Eksepsi dalam JavaScript

Pendahuluan

Dalam pengembangan aplikasi menggunakan JavaScript dan Node.js, memastikan validitas aliran data adalah hal yang krusial. Dua mekanisme utama yang sering digunakan untuk menangani kesalahan adalah asersi (*assertion*) dan eksepsi (*exception handling*). Meskipun kedua contoh kode fungsi `divide(a, b)` di atas sama-sama berfungsi mencegah pembagian dengan nol atau tipe data yang salah, keduanya memiliki filosofi dan tujuan penggunaan yang sangat berbeda di lingkungan produksi.

Asersi untuk Bug Internal

Asersi, yang diimplementasikan pada kode pertama menggunakan modul bawaan `assert`, dirancang untuk memeriksa kondisi yang **seharusnya tidak pernah terjadi** jika program ditulis dengan benar. Ini adalah alat bantu pengembang (*developer tool*) untuk memvalidasi asumsi logika selama fase pengembangan dan pengujian (*testing*).

Jika asersi pada fungsi `divide` gagal, artinya terdapat cacat logika (*bug*) di dalam kode kita sendiri yang mengirimkan argumen salah. Kegagalan asersi akan langsung menghentikan program secara paksa (*fail-fast*). Di banyak bahasa pemrograman, asersi dinonaktifkan di lingkungan produksi demi performa karena fungsinya murni untuk mendeteksi kesalahan internal selama masa pengembangan.

Eksepsi untuk Ketidakpastian Runtime

Sebaliknya, eksepsi menggunakan blok `if` dan `throw` pada kode kedua dirancang untuk menangani kondisi kesalahan yang **dapat diprediksi namun tidak dapat dihindari** saat aplikasi berjalan (*runtime*). Eksepsi mengantisipasi faktor eksternal di luar kendali kode kita, seperti input langsung dari pengguna atau kegagalan koneksi jaringan.

Melalui struktur `try...catch`, program tidak akan mati secara paksa saat error terjadi. Sistem diberikan kesempatan untuk menangkap error tersebut, mencatatnya ke log, dan memberikan respons yang aman atau ramah kepada pengguna.

Kesimpulan

Kita tidak harus memilih salah satu secara mutlak, melainkan harus mengombinasikannya. Asersi digunakan untuk memeriksa *invariants* atau kontrak internal antar fungsi tepercaya selama masa pengujian. Sementara itu, eksepsi wajib digunakan ketika fungsi bertindak sebagai API publik yang menerima input mentah dari luar.

Untuk fungsi `divide(a, b)` di atas, jika ia menerima input langsung dari pengguna atau sistem luar, pendekatan kedua menggunakan eksepsi dan `try...catch` adalah pilihan yang tepat untuk menjaga aplikasi tetap berjalan stabil.

index.htmlindex.js TM KPL M5index.cssindex.js TM M6

TM M6 > JS index.js > ...

```
1 function is_not_fizzbuzz(number) {
2   if (!Number.isInteger(number)) {
3     throw new TypeError("Input harus berupa bilangan bulat");
4   }
5   if (number % 3 === 0 || number % 5 === 0) {
6     return false;
7   }
8   return true;
9 }
10
11 try {
12   console.log(is_not_fizzbuzz(1)); // Output: true
13   console.log(is_not_fizzbuzz(3)); // Output: false
14   console.log(is_not_fizzbuzz(5)); // Output: false
15   console.log(is_not_fizzbuzz(30)); // Output: false
16   console.log(is_not_fizzbuzz(7)); // Output: true
17   console.log(is_not_fizzbuzz(null));
18   console.log(is_not_fizzbuzz(NaN));
19   console.log(is_not_fizzbuzz(Infinity));
20 } catch (error) {
  console.error("Error:", error.message);
}
```

PROBLEMSDEBUG CONSOLEOUTPUTTERMINALPORTS

powershell - TM M6

PS C:\Users\LENOVO\OneDrive\Desktop\TP KPL> cd "TM M6"

PS C:\Users\LENOVO\OneDrive\Desktop\TP KPL\TM M6> node index.js

true

false

false

false

true

Error: Input harus berupa bilangan bulat

PS C:\Users\LENOVO\OneDrive\Desktop\TP KPL\TM M6>